

Assignment 1: Evaluate Regression and Classifier Metrics

This notebook evaluates simple regression and classification baselines on the Iris data set. The purpose is to compare models on held-out data and interpret what the metrics say about predictive performance. The models are intentionally simple, so the analysis focuses on evaluation quality rather than model sophistication.

Executive Summary

This analysis evaluates simple regression and classification baselines on the Iris data set using a stratified train-test split. The better sepal-width regressor was the training-set mean of petal length because it had lower ME magnitude, MAPE, MAE, and MSE than the sepal-length-minus-petal-width estimator. The better classifier was the second sepal-length quantile rule because it had slightly higher accuracy and macro F1 than the first rule. Both winning models should still be treated as baselines rather than strong predictive models because they use very limited information from the available measurements. That limitation matters because predictive models should be judged by how well they generalize to unseen observations, not by whether they appear reasonable on the training data alone (James et al., 2023).

Method Notes

The assignment prompt provides the modeling structure, but two implementation details need to be handled carefully. First, the second regression estimator uses petal width, so the training frame must retain all four Iris feature columns. Second, the second classifier computes the 50th and 75th percentiles, so this notebook applies those cutoffs to the second classifier rather than reusing the first classifier's 25th and 50th percentile cutoffs. These adjustments preserve the assignment's analytical intent while keeping the notebook executable and reproducible.

```
In [1]: import numpy as np
import pandas as pd

from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    precision_recall_fscore_support,
)
```

```
pd.set_option("display.precision", 4)
```

Load and Prepare the Iris Data

The Iris data set includes four flower measurements and a three-class species label. The engineered feature requested in the assignment is created from the ratio of sepal area to petal area. It is retained in the data set, although the specified baseline models do not use it directly.

```
In [2]: iris_raw = datasets.load_iris()

iris = pd.DataFrame(iris_raw.data, columns=iris_raw.feature_names)
iris["type"] = iris_raw.target.astype(int)
iris["new"] = (
    iris["sepal length (cm)"] * iris["sepal width (cm)"]
) / (
    iris["petal length (cm)"] * iris["petal width (cm)"]
)

iris.head()
```

```
Out[2]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	type	new
0	5.1	3.5	1.4	0.2	0	63.7500
1	4.9	3.0	1.4	0.2	0	52.5000
2	4.7	3.2	1.3	0.2	0	57.8462
3	4.6	3.1	1.5	0.2	0	47.5333
4	5.0	3.6	1.4	0.2	0	64.2857

Train-Test Split

The split uses 80% of the observations for training and 20% for testing. Stratification keeps the three species classes balanced in the test set, which makes the classification metrics easier to interpret. This train-test structure follows the general statistical learning principle that model performance should be evaluated on data not used to fit the model (James et al., 2023).

```
In [3]: feature_cols = iris_raw.feature_names
X = iris[feature_cols]
y = iris["type"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```

test_size=0.2,
random_state=42,
stratify=y,
)

split_summary = pd.DataFrame({
    "Partition": ["Training", "Test"],
    "Rows": [len(X_train), len(X_test)],
    "Setosa": [(y_train == 0).sum(), (y_test == 0).sum()],
    "Versicolor": [(y_train == 1).sum(), (y_test == 1).sum()],
    "Virginica": [(y_train == 2).sum(), (y_test == 2).sum()],
})

split_summary

```

```

Out[3]:

```

	Partition	Rows	Setosa	Versicolor	Virginica
0	Training	120	40	40	40
1	Test	30	10	10	10

Regression Metrics for Sepal Width

The regression task evaluates two constant estimators for sepal width on the test set. Mean error shows directional bias, while MAE and MSE summarize error magnitude. MAPE translates the error into a percentage of the observed sepal width. This percentage interpretation is useful here because sepal width is never near zero, although percentage-error measures can become unstable when actual values approach zero (Hyndman & Koehler, 2006).

```

In [4]: def regression_metrics(actual, predicted):
    actual = np.asarray(actual, dtype=float)
    predicted = np.asarray(predicted, dtype=float)
    error = actual - predicted
    return {
        "ME": np.mean(error),
        "MPE": np.mean(error / actual) * 100,
        "MAPE": np.mean(np.abs(error / actual)) * 100,
        "MAE": np.mean(np.abs(error)),
        "MSE": np.mean(error ** 2),
    }

actual_sepal_width = X_test["sepal width (cm)"]

est1_value = np.mean(X_train["petal length (cm)"])
est2_value = np.mean(X_train["sepal length (cm)"] - X_train["petal width (cm)"])

regression_results = pd.DataFrame([
    {
        "Estimator": "Mean training petal length",
        "Prediction Value": est1_value,
        **regression_metrics(actual_sepal_width, np.repeat(est1_value, len(actual_s

```

```

    },
    {
        "Estimator": "Mean training sepal length minus petal width",
        "Prediction Value": est2_value,
        **regression_metrics(actual_sepal_width, np.repeat(est2_value, len(actual_s
    },
])

regression_results

```

Out[4]:

	Estimator	Prediction Value	ME	MPE	MAPE	MAE	MSE
0	Mean training petal length	3.7700	-0.6767	-23.7499	24.1859	0.6940	0.6018
1	Mean training sepal length minus petal width	4.6367	-1.5433	-52.1981	52.1981	1.5433	2.5258

The training-set mean of petal length is the better regression estimator. It has smaller absolute error, smaller percentage error, and smaller squared error. Both estimators overpredict sepal width because the mean errors are negative when error is calculated as actual minus predicted, but the second estimator overpredicts much more severely.

Classification Metrics for Species Type

The classifiers use sepal-length quantiles from the training data as simple decision rules. Accuracy gives the overall proportion of correct labels, while precision, recall, and F1 show whether the classifiers behave differently across classes. That distinction matters because a multiclass classifier can have similar overall accuracy while still performing unevenly across individual classes (Sokolova & Lapalme, 2009).

```

In [5]: def quantile_classifier(sepal_length, cutoffs):
    predictions = np.zeros(len(sepal_length), dtype=int)
    predictions[sepal_length > cutoffs[0]] = 1
    predictions[sepal_length > cutoffs[1]] = 2
    return predictions

est3 = np.percentile(X_train["sepal length (cm)"], [25, 50])
est4 = np.percentile(X_train["sepal length (cm)"], [50, 75])

y_hat_1 = quantile_classifier(X_test["sepal length (cm)"], est3)
y_hat_2 = quantile_classifier(X_test["sepal length (cm)"], est4)

classification_rows = []
for name, cutoffs, predictions in [
    ("Classifier 1: 25th and 50th percentile cutoffs", est3, y_hat_1),
    ("Classifier 2: 50th and 75th percentile cutoffs", est4, y_hat_2),
]:
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test,
        predictions,

```

```

        average="macro",
        zero_division=0,
    )
    classification_rows.append({
        "Classifier": name,
        "Cutoff 1": cutoffs[0],
        "Cutoff 2": cutoffs[1],
        "Accuracy": accuracy_score(y_test, predictions),
        "Macro Precision": precision,
        "Macro Recall": recall,
        "Macro F1": f1,
    })

classification_results = pd.DataFrame(classification_rows)
classification_results

```

Out[5]:

	Classifier	Cutoff 1	Cutoff 2	Accuracy	Macro Precision	Macro Recall	Macro F1
0	Classifier 1: 25th and 50th percentile cutoffs	5.10	5.75	0.5667	0.5453	0.5667	0.5263
1	Classifier 2: 50th and 75th percentile cutoffs	5.75	6.40	0.6000	0.5780	0.6000	0.5825

In [6]: labels = list(iris_raw.target_names)

```

for name, predictions in [
    ("Classifier 1", y_hat_1),
    ("Classifier 2", y_hat_2),
]:
    print(name)
    print("Confusion matrix")
    print(pd.DataFrame(confusion_matrix(y_test, predictions), index=labels, columns=labels))
    print("\nClassification report")
    print(classification_report(y_test, predictions, target_names=labels, zero_divi

```

Classifier 1

Confusion matrix

	setosa	versicolor	virginica
setosa	5	4	1
versicolor	2	2	6
virginica	0	0	10

Classification report

	precision	recall	f1-score	support
setosa	0.71	0.50	0.59	10
versicolor	0.33	0.20	0.25	10
virginica	0.59	1.00	0.74	10
accuracy			0.57	30
macro avg	0.55	0.57	0.53	30
weighted avg	0.55	0.57	0.53	30

Classifier 2

Confusion matrix

	setosa	versicolor	virginica
setosa	9	1	0
versicolor	4	3	3
virginica	0	4	6

Classification report

	precision	recall	f1-score	support
setosa	0.69	0.90	0.78	10
versicolor	0.38	0.30	0.33	10
virginica	0.67	0.60	0.63	10
accuracy			0.60	30
macro avg	0.58	0.60	0.58	30
weighted avg	0.58	0.60	0.58	30

The second classifier performs slightly better, with accuracy of 0.60 and macro F1 of 0.58 compared with 0.57 accuracy and 0.53 macro F1 for the first classifier. The difference is small, and neither classifier should be treated as strong. The class-level results show that versicolor is the hardest class for these one-variable rules to separate, which is why the class-level precision, recall, and F1 values are more informative than accuracy alone (Sokolova & Lapalme, 2009).

Interpretation and Improvement Plan

The better regressor is the training-set petal-length mean because it produces lower ME magnitude, MAPE, MAE, and MSE. The better classifier is the second quantile rule because it has higher accuracy and macro F1. However, both comparisons are limited because the models are baselines rather than serious predictive models.

A stronger next step would use all four Iris measurements and evaluate models with cross-validation. For regression, linear regression, regularized regression, or tree-based regression could be compared for sepal-width prediction. For classification, multinomial logistic regression, k-nearest neighbors, classification trees, support vector machines, or ensemble methods would be more appropriate than a single sepal-length rule. The engineered ratio feature should be tested rather than assumed useful, because an engineered feature can improve performance only when it adds information that generalizes to held-out data.

References

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679-688.

<https://doi.org/10.1016/j.ijforecast.2006.03.001>

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: With applications in Python*. Springer. <https://doi.org/10.1007/978-3-031-38747-0>

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427-437.

<https://doi.org/10.1016/j.ipm.2009.03.002>